



Exercice 1 – Guide pas-à-pas (Auth simple)

Objectif : Implémenter pas-à-pas un mini système d'authentification (signup → login → middleware) sans te donner tout le code d'un coup.

1. Initialisation & structure

- Crée le dossier `auth-api`, `npm init -y` et installe `express`, `mongoose`, `dotenv`, `bcrypt`, `jsonwebtoken`.
- Crée `app.js`, `server.js`, dossiers `models`, `routes`, `middleware`.

2. Connexion MongoDB

- Dans `.env` : `MONGO_URI`, `JWT_SECRET`, `PORT`.
- Dans `app.js`, charge `dotenv`, connecte `mongoose` et démarre `Express`.

3. Modèle User

- Dans `models/User.js`, définis un schéma `Mongoose` avec
 1. `username: String, required, unique`
 2. `password: String, required`

4. Route Signup

- Dans `routes/auth.js`, crée `POST /signup`
- Récupère `username` & `password` dans `req.body`
- Hash le mot de passe avec `bcrypt` avant de créer l'utilisateur
- Renvoie `201` ou erreur

5. Route Login

- Ajoute `POST /login`
- Recherche l'utilisateur par `username`

- Compare le password ; si OK, signe un JWT (`userId` dans le payload)
- Renvoie `{ token }` ou `401`

6. Middleware d'auth

- Dans `middleware/auth.js`, récupère l'en-tête `Authorization`
- Vérifie (`jwt.verify`) et extrait `userId`
- Si valide, `req.userId = ...`, sinon `401`

7. Test Progressif

- Teste **seul** chaque étape dans Postman :
 1. Signup → nouvel user
 2. Login → token
 3. Accès à une route protégée (ex. `/private`) avec ton middleware



Exercice 2 – Bibliothèque sécurisée (API complète)

Contexte : Tu gères une petite API de livres, mais seuls les utilisateurs authentifiés peuvent interagir.

But : fusionner gestion d'utilisateurs, vérification d'identifiants, JWT, middleware, et CRUD livres.

Structure

```
bash
CopyEdit
books-auth-api/
├─ models/
│   ├── User.js
│   └── Book.js
├─ routes/
│   ├── auth.js
│   └── books.js
├─ middleware/auth.js
└─ app.js
```

```
| server.js
| .env
```

1.

2. Modèles

- **User** : comme dans l'exercice 1, plus champ `role: { type: String, default: 'user' }`

Book :

```
js
CopyEdit
{
  title: { type: String, required: true },
  author: { type: String, required: true },
  year: Number,
  owner: { type: mongoose.Schema.Types.ObjectId, ref: 'User',
required: true }
}
```

○

3. Routes Auth (/auth)

- `POST /signup` et `POST /login` : copié de l'exercice 1

4. Middleware

- `auth.js` protège toutes les routes suivantes

5. Routes Books (/books)

- **POST /books** :
 - Requier authentication
 - Crée un livre en ajoutant `owner = req.user.id`
- **GET /books** :
 - Renvoie **seulement** les livres dont `owner === req.user.id`
- **GET /books/:id** :

- Vérifie que le livre appartient à l'utilisateur
- **PUT /books/:id & DELETE /books/:id** :
 - Même vérification avant modification/suppression

6. Tests à réaliser

- Plusieurs users : chaque user ne doit voir que ses livres
- Tenter d'accéder/éditer un livre d'un autre user → **403 Forbidden**
- Vérifier expiration de token

Exercices Complet “All-in-One”

Exercice 5 – To-Do List sécurisée

Reprends ta **To-Do API** (Exercice 1) et :

1. Ajoute **inscription & connexion** (signup/login) comme ci-dessus.
2. Chaque tâche doit avoir un champ **owner : ObjectId** (lié à **req.userId**).
3. Protéger **toutes** les routes CRUD (**/tasks**) avec le middleware d'auth.
4. **/tasks** (GET) ne renvoie **que** les tâches de l'utilisateur connecté.
5. Teste avec Postman : plusieurs utilisateurs, tâches séparées.

Exercice 6 – API de Notes collaboratives

Tout autre sujet : tu construis une API de **notes** où :

1. Les utilisateurs s'inscrivent et se connectent (comme Exercice 1).
2. Ils peuvent créer, modifier et supprimer des **documents collaboratifs (Note)** :
 - Titre, contenu, liste de **collaborators : [ObjectId]**

3. Seuls le **propriétaire** ou un **collaborateur** peut lire ou modifier une note.
4. Ajoute une route `POST /notes/:id/share` où le propriétaire peut ajouter d'autres `userId` comme collaborateurs.
5. Protège toutes les routes avec auth + vérification de droit d'accès.